

Background of Fully Homomorphic Encryption

CKKS and TFHE

HU Qi

The University of Hong Kong

November 17, 2025

1.1 Motivation and Definition of FHE

- ▶ **Reality tension:** data must be “used” without ever being “seen”.
 - ▶ Cloud outsourcing: model inference, statistical analytics, search indexing.
 - ▶ Regulatory pressure: GDPR, HIPAA, and cross-border data controls.
 - ▶ Traditional encryption covers storage and transit, but requires decryption for computation.
- ▶ **Privacy-computing goal:** make data usable while keeping it confidential.
 - ▶ Servers only handle ciphertext yet complete the agreed computation.
 - ▶ Clients decrypt and recover the same outcome as plaintext evaluation.
- ▶ **FHE definition:** core property $\text{Dec}(f(\text{Enc}(x))) = f(x)$.
 - ▶ Meaning: homomorphic f on ciphertext, decrypt for the plaintext result.
- ▶ **Typical applications:**
 - ▶ Healthcare: encrypted imaging or genomics analysed in the cloud, hospitals decrypt locally.
 - ▶ Finance: encrypted transactions scored for risk or collaborative anti-fraud.
 - ▶ Privacy smart contracts: on-chain ciphertexts with off-chain/on-chain FHE execution.

Background of Fully Homomorphic Encryption

└ 1.1 Motivation and Definition of FHE

Start with the question: why compute under encryption? Cloud inference, analytics, and risk scoring clash with GDPR, HIPAA, and cross-border rules that forbid revealing plaintext. Traditional encryption protects storage and transit yet forces decryption right before computation—the riskiest moment. Privacy computing keeps servers on ciphertext while clients decrypt to recover the same output; the identity $\text{Dec}(f(\text{Enc}(x))) = f(x)$ uses the same logical f , with ciphertexts processed through matching homomorphic operations. FHE keeps data encrypted end-to-end, removing the server-side decrypt-compute step and shrinking leakage on untrusted infrastructure. Applications span encrypted medical AI, financial risk scoring on ciphertexts, encrypted inputs to cloud models with local decryption, and privacy smart contracts.

1.1 Motivation and Definition of FHE

- **Reality tension:** data must be "used" without ever being "seen".
 - Cloud outsourcing: model inference, statistical analytics, scoring indexing.
 - Regulatory pressure: GDPR, HIPAA, and cross-border data controls.
 - Traditional encryption covers storage and transit, but requires decryption for computation.
- **Privacy-computing goal:** make data usable while keeping it confidential.
 - Servers only handle ciphertext yet complete the agreed computation.
 - Clients decrypt and recover the same outcome as plaintext evaluation.
- **FHE definition:** core property $\text{Dec}(f(\text{Enc}(x))) = f(x)$.
 - Meaning: homomorphic f on ciphertext, decrypt for the plaintext result.
- **Typical applications:**
 - Healthcare: encrypted imaging or genomics analyzed in the cloud, hospitals decrypt locally.
 - Finance: encrypted transactions scored for risk or collaborative anti-fraud.
 - Privacy smart contracts: on-chain ciphertexts with off-chain/on-chain FHE execution.

1.2 Evolution of Fully Homomorphic Encryption

- ▶ 2009: Gentry's breakthrough first-generation scheme based on ideal lattices.
- ▶ 2011–2014: Second generation (BGV, BFV) improves efficiency via leveled FHE and bootstrapping optimizations.
- ▶ 2016: CKKS introduces approximate arithmetic for real-number workloads such as machine learning inference.
- ▶ 2016+: TFHE enables fast gate-by-gate Boolean computation with rapid bootstrapping.
- ▶ Current landscape: hybrid approaches, GPU acceleration, and practical libraries (HElib, SEAL, Concrete, OpenFHE).

2025-11-17

Background of Fully Homomorphic Encryption

└ 1.2 Evolution of Fully Homomorphic Encryption

Here I sketch the trajectory of FHE. After Gentry's 2009 construction, leveled schemes like BGV and BFV reduced overhead. CKKS later delivered approximate arithmetic suited for ML, while TFHE focused on fast Boolean gates with efficient bootstrapping. Today the ecosystem blends these ideas, pushing toward practical deployments with optimized libraries and hardware acceleration.

1.2 Evolution of Fully Homomorphic Encryption

- ▶ 2009: Gentry's breakthrough first-generation scheme based on ideal lattices.
- ▶ 2011-2014: Second generation (BGV, BFV) improves efficiency via leveled FHE and bootstrapping optimizations.
- ▶ 2016: CKKS introduces approximate arithmetic for real-number workloads such as machine learning inference.
- ▶ 2016-: TFHE enables fast gate-by-gate Boolean computation with rapid bootstrapping.
- ▶ Current landscape: hybrid approaches, GPU acceleration, and practical libraries (HElib, SEAL, Concrete, OpenFHE).

2. Mathematical Foundations Overview

- ▶ Modular arithmetic and polynomial rings ($\mathbb{Z}_q, R_q$) as the container for ciphertext addition/multiplication.
- ▶ Batched (SIMD) encoding intuition: one encrypted polynomial carries many data slots.
- ▶ Hardness assumptions from LWE/RLWE tying noisy linear equations to ring-based constructions.

Preview the three ingredients: the modular polynomial ring container, SIMD-style batching intuition, and the LWE/RLWE hardness assumptions that secure schemes.

- ▶ Modular arithmetic and polynomial rings ($\mathbb{Z}_q, R_q$) as the container for ciphertext addition/multiplication.
- ▶ Batched (SIMD) encoding intuition: one encrypted polynomial carries many data slots.
- ▶ Hardness assumptions from LWE/RLWE tying noisy linear equations to ring-based constructions.

2025-11-17

- ## Background of Fully Homomorphic Encryption

2.1 Modular Arithmetic and Polynomial Ring

- Modern FHE schemes encode ciphertexts as polynomials over rings to support both addition and multiplication (ring = additive group + multiplicative semigroup).
- Integer modular ring: $\mathbb{Z}_q = \{0, 1, \dots, q-1\}$ with addition/multiplication taken mod q (typically a large prime or an RNS prime chain).
- Polynomial ring: $R_q = \mathbb{Z}_q[x]/(x^N + 1)$ with N usually a power of two.
 - Elements are polynomials of degree $< N$ with coefficients in $\mathbb{Z}_q$.
 - Multiplication: convolve coefficients, then reduce by $(x^N + 1)$ so $x^N \equiv -1$.

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

2025-11-17

- ## Background of Fully Homomorphic Encryption

└ 2.2 Intuition for Batched (SIMD) Encoding

Explain that batching in CKKS means taking a complex vector z and embedding it into the polynomial ring. The key fact is the approximate isomorphism $R_q \otimes \mathbb{C} \cong \mathbb{C}^{N/2}$ when N is a power of two; hence each ciphertext holds $N/2$ complex slots. Practically we run an inverse DFT on z in the complex domain to produce $m(x)$, then scale and round to an integer polynomial $\tilde{m}(x)$ and reduce modulo q so it lives in $\mathbb{Z}_q[x]$ before encryption.

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

2.2 Intuition for Batched (SIMD) Encoding

- Goal: start from a complex vector

$$x = [x_1, x_2, \dots, x_{N/2}] \in \mathbb{C}^{N/2},$$
 encode it into the CKKS ring $R_q = \mathbb{Z}_q[x]/(x^N + 1)$.
- Observation: when N is a power of two, there is an approximate isomorphism

$$R_q \otimes \mathbb{C} \cong \mathbb{C}^{N/2},$$
 so one ciphertext carries $N/2$ complex slots.
- Practical encoding: run the complex-domain inverse DFT (discrete Fourier transform) to map slots into polynomial coefficients

$$m(x) = \text{Inverse-DFT}(z),$$
 then scale and round to obtain an integer polynomial $\hat{m}(x)$ before reducing modulo q , so $\hat{m}(x) \bmod q \in \mathbb{Z}_q[x]$.

► Practical encoding: run the complex-domain inverse DFT (discrete Fourier transform) to map slots into polynomial coefficients

$$m(x) = \text{Inverse-DFT}(x),$$

then scale and round to obtain an integer polynomial $\tilde{m}(x)$ before reducing modulo q , so $\tilde{m}(x) \bmod q \in \mathbb{Z}_q[x]$.

- Practical encoding: run the complex-domain inverse DFT (discrete Fourier transform) to map slots into polynomial coefficients

$$m(x) = \text{Inverse-DFT}(z),$$

then scale and round to obtain an integer polynomial $\tilde{m}(x)$ before reducing modulo q , so $\tilde{m}(x) \bmod q \in \mathbb{Z}_q[x]$.

2.3 LWE and RLWE

- ▶ Motivation: Learning With Errors (LWE) and Ring-LWE (RLWE) provide computationally controllable, provably secure public-key and homomorphic encryption foundations.
- ▶ Core idea: add random noise to hide plaintexts while preserving ring/polynomial structure for computation, reducing security to GapSVP (quantum-resistant).
- ▶ LWE (vector form): samples $(A, b = As + e \bmod q)$ where A is random, s secret, e small noise.
- ▶ RLWE (polynomial form): samples $(a(x), b(x) = a(x)s(x) + e(x) \bmod q)$ over R_q .
- ▶ Transition: RLWE inherits LWE hardness while enabling structured polynomials that FHE schemes rely on.

Background of Fully Homomorphic Encryption

- Mathematical Foundations of FHE

2.3 LWE and RLWE

Introduce why we need LWE/RLWE: they add controlled random noise to hide plaintexts while preserving the ring/polynomial structure needed for computation, with security reducible to GapSVP even against quantum adversaries. Briefly define LWE via noisy linear equations $b = As + e \pmod{q}$ and RLWE via polynomial samples $b(x) = a(x)s(x) + e(x) \pmod{q}$ inside R_q . Emphasize that RLWE inherits LWE hardness yet aligns with the polynomial ring container used in FHE, enabling efficient and analyzable homomorphic schemes.

- Motivation: Learning With Errors (LWE) and Ring-LWE (RLWE) provide computationally controllable, provably secure public-key and homomorphic encryption foundations.
- Core idea: add random noise to hide plaintexts while preserving ring/polynomial structure for computation, reducing security to GapSVP (quantum-resistant).
- LWE (vector form): samples $(A, b) = (A, b = A \cdot s + e \bmod q)$ where A is random, s secret, e small noise.
- RLWE (polynomial form): samples $(a(x), b(x) = a(x) \cdot s(x) + e(x) \bmod q)$ over R_q .
- Transition: RLWE inherits LWE hardness while enabling structured polynomials that FHE schemes rely on.

2.4 LWE: Key Generation, Encryption, Decryption

Key generation Sample $s \leftarrow \mathbb{Z}_q^n$, noise $e \leftarrow \mathbb{Z}_q^n$, and public randomness $A \leftarrow \mathbb{Z}_q^{n \times n}$. Set

$$b = As + e \pmod q.$$

Publish (A, b) ; keep s secret.

Encryption of $m \in \{0, 1\}$ Pick a small random vector r , then compute

$$c_0 = A^T r, \quad c_1 = b^T r + \Delta m,$$

forming ciphertext $c = (c_0, c_1)$, where the scaling constant is $\Delta = q/2$ for a binary alphabet (or q/L for L equally spaced levels). As a quick sanity check, let $r = \mathbf{1}$ so c_0 simply sums the columns of A .

Decryption with s Evaluate

$$c_1 - c_0^T s = e^T r + \Delta m.$$

Because $e^T r$ is tiny, we recover the message by normalizing and rounding:

$$\hat{m} = \text{round}\left(\frac{c_1 - c_0^T s}{\Delta}\right),$$

which snaps back to the original m .

2025-11-17

- Background of Fully Homomorphic Encryption
 - Mathematical Foundations of FHE
 - 2.4 LWE: Key Generation, Encryption, Decryption

“Here is the full LWE workflow. First sample the bright-violet secret key s along with noise e and public matrix A , then publish the vector $b = As + e \pmod q$ as part of the public key. The bright-orange m is the plaintext bit we encrypt. Pick a short random vector r —in the simplest thought experiment let $r = \mathbf{1}$ so $c_0 = A^T r$ just sums the columns—and compute c_0 plus $c_1 = b^T r + \Delta m$ where $\Delta = q/2$ because the message alphabet has two symbols (if there were L symbols we’d set $\Delta = q/L$). Decryption is $c_1 - c_0^T s$: the cross terms cancel, leaving $e^T r + \Delta m$. Since the error is tiny, divide by Δ and round to recover m . This emphasizes how the noisy value sits near a multiple of Δ and snaps back to the intended plaintext.”

2.4 LWE: Key Generation, Encryption, Decryption

Key generation Sample $s \leftarrow \mathbb{Z}_q^n$, noise $e \leftarrow \mathbb{Z}_q^n$, and public randomness $A \leftarrow \mathbb{Z}_q^{n \times n}$. Set $b = As + e \pmod q$.

Publish (A, b) ; keep s secret.

Encryption of $m \in \{0, 1\}$ Pick a small random vector r , then compute $c_0 = A^T r$, $c_1 = b^T r + \Delta m$, forming ciphertext $c = (c_0, c_1)$ where the scaling constant is $\Delta = q/2$ for a binary alphabet (or q/L for L equally spaced levels). As a quick sanity check, let $r = \mathbf{1}$ so c_0 simply sums the columns of A .

Decryption with s Evaluate $c_1 - c_0^T s = e^T r + \Delta m$. Because $e^T r$ is tiny, we recover the message by normalizing and rounding: $\hat{m} = \text{round}\left(\frac{c_1 - c_0^T s}{\Delta}\right)$, which snaps back to the original m .

2.5 RLWE: $b(x) = a(x)s(x) + e(x) \pmod q$

Ring setting Work in $R_q = \mathbb{Z}_q[x]/(x^N + 1)$ with secret $s(x)$ and small error $e(x)$ drawn from a narrow distribution on R_q .

Key generation Sample public $a(x) \leftarrow R_q$, draw $e(x)$, and set

$$b(x) = a(x)s(x) + e(x) \pmod q.$$

Publish $(a(x), b(x))$, keep $s(x)$ secret.

Encryption of plaintext $m(x) \in R_q$ Pick a small, sparse $r(x)$ (conceptually set $r(x) = 1$ to sanity-check the formulas) and output

$$c_0(x) = a(x)r(x), \quad c_1(x) = b(x)r(x) + \Delta m(x),$$

with $\Delta = q/t$ for t plaintext slots (e.g., $t = 2$ for binary messages), then package $c(x) = (c_0(x), c_1(x))$.

Decryption Compute

$$c_1(x) - c_0(x)s(x) = \Delta m(x) + \tilde{e}(x),$$

divide by Δ , and round coefficient-wise to strip the small noise.

2025-11-17

Background of Fully Homomorphic Encryption

- Mathematical Foundations of FHE
- 2.5 RLWE: $b(x) = a(x)s(x) + e(x) \pmod q$

“RLWE lifts everything into the polynomial ring $R_q = \mathbb{Z}_q[x]/(x^N + 1)$. The bright-violet $s(x)$ is now a polynomial secret with small coefficients; noise polynomials come from discrete Gaussian or bounded distributions on the ring. Key generation samples a public $a(x)$, draws $e(x)$, and publishes $b(x) = a(x)s(x) + e(x) \pmod q$. To encrypt a bright-orange $m(x)$ we pick a fresh small $r(x)$ —even imagining $r(x) = 1$ for intuition—and set $c_0 = ar$ and $c_1 = br + \Delta m(x)$ with $\Delta = q/t$ depending on how many plaintext slots we encode. Decryption subtracts $c_0s(x)$ from c_1 : the as terms cancel, leaving $\Delta m(x)$ plus tiny noise. Dividing by Δ and rounding coefficient-wise recovers $m(x)$.

2.5 RLWE: $b(x) = a(x)s(x) + e(x) \pmod q$

Ring setting Work in $R_q = \mathbb{Z}_q[x]/(x^N + 1)$ with secret $s(x)$ and small error $e(x)$ drawn from a narrow distribution on R_q .

Key generation Sample public $a(x) \leftarrow R_q$, draw $e(x)$, and set $b(x) = a(x)s(x) + e(x) \pmod q$.

Publish $(a(x), b(x))$, keep $s(x)$ secret.

Encryption of plaintext $m(x) \in R_q$ Pick a small, sparse $r(x)$ (conceptually set $r(x) = 1$ to sanity-check the formulas) and output $c_0(x) = a(x)r(x), \quad c_1(x) = b(x)r(x) + \Delta m(x)$, with $\Delta = q/t$ for t plaintext slots (e.g., $t = 2$ for binary messages), then package $c(x) = (c_0(x), c_1(x))$

Decryption Compute $c_1(x) - c_0(x)s(x) = \Delta m(x) + \tilde{e}(x)$, divide by Δ , and round coefficient-wise to strip the small noise.

3. CKKS Overview

- ▶ Approximate-number FHE supporting packed real/complex vectors (SIMD-style).
- ▶ Workflow: scale plaintexts by a large Δ , encode integers into RLWE polynomials, then encrypt.
- ▶ Homomorphic addition is cheap (same scale); multiplication requires relinearization + rescale to control growth.
- ▶ Resulting ciphertexts keep bounded precision error, making CKKS practical for ML inference and analytics.

Background of Fully Homomorphic Encryption

3. CKKS Overview

“This section introduces CKKS, currently the most widely deployed homomorphic encryption scheme for real and complex vectors. It lets us perform additions and multiplications directly on encrypted data while keeping the final precision error under control. The core trick is to multiply real numbers by a large scaling factor Δ , convert them to integers, and encode those integers into RLWE polynomials before encrypting. Additions preserve the scale so they are almost free. Multiplications, however, inflate the scale and ciphertext size, so we fix things with two steps: relinearization to bring the ciphertext back to its normal dimension, and rescaling to divide away the oversized scale so precision does not keep degrading.”

- ▶ Approximate-number FHE supporting packed real/complex vectors (SIMD-style).
- ▶ Workflow: scale plaintexts by a large Δ , encode integers into RLWE polynomials, then encrypt.
- ▶ Homomorphic addition is cheap (same scale); multiplication requires relinearization + rescale to control growth.
- ▶ Resulting ciphertexts keep bounded precision error, making CKKS practical for ML inference and analytics.

3.1 CKKS Encoding and Scaling

Goal Encode a real vector $(z_1, z_2, \dots, z_k)$ into the polynomial ring $R_q = \mathbb{Z}_q[x]/(x^N + 1)$ so CKKS can process it homomorphically.

Scaling Multiply each real by a large Δ so fractional parts become big integers:

$$\tilde{m} = \Delta \cdot m,$$

because all CKKS arithmetic happens modulo q .

Encoding (Same SIMD trick as Section 2.2) leverage $R_q \otimes \mathbb{C} \cong \mathbb{C}^{N/2}$ to pack the scaled vector into one polynomial—each slot stores one entry, enabling parallel processing.

Decoding After decryption, recover the integer vector and reverse the scaling:

$$m \approx \frac{1}{\Delta} \cdot \tilde{m},$$

yielding the original reals up to a controllable approximation error.

2025-11-17

Background of Fully Homomorphic Encryption

- CKKS
- 3.1 CKKS Encoding and Scaling

“To let CKKS encrypt and compute on reals we first have to move them into the integer polynomial ring. Two steps make this work: scaling and encoding. We multiply each real number by a large Δ , turning fractional values into big integers, because all homomorphic operations happen modulo q and floats are not allowed. Then we rely on the ring decomposition $R_q \otimes \mathbb{C} \cong \mathbb{C}^{N/2}$ to map an entire vector into a single polynomial—each coefficient, or slot, holds one value, exactly like the SIMD batching we introduced earlier. This is why one CKKS ciphertext can process many numbers in parallel. During decryption we reverse the process: convert the polynomial back to its integer vector and divide by Δ to get an approximation of the original m . Hence CKKS is an approximate FHE scheme with controllable precision error.”

3.1 CKKS Encoding and Scaling

Goal Encode a real vector $(z_1, z_2, \dots, z_k)$ into the polynomial ring $R_q = \mathbb{Z}_q[x]/(x^N + 1)$ so CKKS can process it homomorphically.

Scaling Multiply each real by a large Δ so fractional parts become big integers:

$$\tilde{m} = \Delta \cdot m,$$

because all CKKS arithmetic happens modulo q .

Encoding (Same SIMD trick as Section 2.2) leverage $R_q \otimes \mathbb{C} \cong \mathbb{C}^{N/2}$ to pack the scaled vector into one polynomial—each slot stores one entry, enabling parallel processing.

Decoding After decryption, recover the integer vector and reverse the scaling:

$$m \approx \frac{1}{\Delta} \cdot \tilde{m},$$

yielding the original reals up to a controllable approximation error.

3.2 CKKS Encryption and Decryption

- Plaintext prep** Before encryption, scale by Δ and encode the real vector into an integer polynomial $m(x)$ (Section 2.2 + 3.1).
- Key generation** Identical to RLWE (Section 2.5): pick small $s(x)$, sample random $a(x)$ and noise $e(x)$, then publish

$$b(x) = a(x)s(x) + e(x) \pmod{q}.$$

- Encryption** Form ciphertext $c(x) = (c_0(x), c_1(x))$ via
$$c_0(x) = a(x)r(x), \quad c_1(x) = b(x)r(x) + \Delta m(x),$$
where $r(x)$ is a fresh small polynomial.
- Decryption** Subtract and divide by the recorded scale:

$$c_1(x) - c_0(x)s(x) = \Delta m(x) + \tilde{e}(x)$$

and compute $m(x) \approx \frac{1}{\Delta}(c_1 - c_0s)$.

- Scale tracking** CKKS ciphertexts explicitly store the current Δ : here it records fixed-point precision (unlike RLWE's $\Delta = q/t$ embedding) and is updated after every homomorphic operation.

2025-11-17

- Background of Fully Homomorphic Encryption
 - CKKS
 - 3.2 CKKS Encryption and Decryption

3.2 CKKS Encryption and Decryption

Plaintext prep Before encryption, scale by Δ and encode the real vector into an integer polynomial $m(x)$ (Section 2.2 + 3.1).

Key generation Identical to RLWE (Section 2.5): pick small $s(x)$, sample random $a(x)$ and noise $e(x)$, then publish

$$b(x) = a(x)s(x) + e(x) \pmod{q}.$$

Encryption Form ciphertext $c(x) = (c_0(x), c_1(x))$ via

$$c_0(x) = a(x)r(x), \quad c_1(x) = b(x)r(x) + \Delta m(x),$$

where $r(x)$ is a fresh small polynomial.

Decryption Subtract and divide by the recorded scale:

$$c_1(x) - c_0(x)s(x) = \Delta m(x) + \tilde{e}(x)$$

and compute $m(x) \approx \frac{1}{\Delta}(c_1 - c_0s)$.

Scale tracking CKKS ciphertexts explicitly store the current Δ : here it records fixed-point precision (unlike RLWE's $\Delta = q/t$ embedding) and is updated after every homomorphic operation.

“This slide revisits encryption and decryption in CKKS. The workflow is literally the same as the RLWE recipe from Section 2.5—the only twist is that CKKS prepares the plaintext by scaling it with Δ and encoding it into an integer polynomial before we ever touch the public key. Key generation samples a small-coefficient secret $s(x)$, random $a(x)$, and noise $e(x)$ to produce $b(x) = a(x)s(x) + e(x) \pmod{q}$. Encryption outputs $c_0 = ar$ and $c_1 = br + \Delta m(x)$ for a fresh small $r(x)$. Decryption subtracts c_0s from c_1 so the RLWE terms cancel, leaving $\Delta m(x)$ plus tiny noise; dividing by the stored Δ recovers the approximate real vector. Crucially, CKKS’s Δ represents the fixed-point scale for numerical precision, whereas RLWE’s $\Delta = q/t$ simply maps a plaintext modulus into q . That’s why every CKKS ciphertext carries its current scale, which must be updated after additions, multiplications, and rescaling to keep precision under control.”

3.3 CKKS Rescaling and Scale Management

Why rescale? Ciphertexts encode scaled approximations $m \cdot \Delta$. A homomorphic multiplication inflates the scale to Δ^2 :

$$(\Delta \cdot m_0) \cdot (\Delta \cdot m_1) = \Delta^2 \cdot (m_0 m_1),$$

so repeated multiplications would push the scale (and noise) out of range.

Operation With ciphertext $c = (c_0, c_1) \in R_Q$ at scale Δ over modulus chain $Q = q_0 q_1 \dots q_L$,

$$c' = \text{Rescale}(c) = \left\lfloor \frac{c}{q_L} \right\rfloor \bmod \frac{Q}{q_L},$$

consuming the bottom modulus.

Effect New parameters become

$$\Delta' = \frac{\Delta}{q_L}, \quad Q' = \frac{Q}{q_L},$$

keeping the fixed-point scale within target bounds at the cost of one level.

2025-11-17

Background of Fully Homomorphic Encryption

- CKKS

- 3.3 CKKS Rescaling and Scale Management

“Rescaling is CKKS’s way of taming the scale explosion caused by multiplication. Remember that ciphertexts store scaled approximations $m \cdot \Delta$; multiplying two of them yields $(m_1 \Delta)(m_2 \Delta) = (m_1 m_2) \Delta^2$, just like fixed-point multiplication shifting the decimal twice. If we do nothing, the scale and noise blow up. The remedy is the rescale step: divide the ciphertext by the lowest modulus q_L in the chain and round, producing $c' = \lfloor c/q_L \rfloor \bmod (Q/q_L)$. This simultaneously reduces the modulus to $Q' = Q/q_L$ and shrinks the scale to $\Delta' = \Delta/q_L$, pulling the fixed-point representation back into the desired range. Every rescale consumes a level, so we plan circuits so there’s enough modulus budget for all future multiplications.”

3.3 CKKS Rescaling and Scale Management

Why rescale? Ciphertexts encode scaled approximations $m \cdot \Delta$. A homomorphic multiplication inflates the scale to Δ^2 :

$$(\Delta \cdot m_0) \cdot (\Delta \cdot m_1) = \Delta^2 \cdot (m_0 m_1),$$

so repeated multiplications would push the scale (and noise) out of range.

Operation With ciphertext $c = (c_0, c_1) \in R_Q$ at scale Δ over modulus chain $Q = q_0 q_1 \dots q_L$,

$$c' = \text{Rescale}(c) = \left\lfloor \frac{c}{q_L} \right\rfloor \bmod \frac{Q}{q_L},$$

consuming the bottom modulus.

Effect New parameters become

$$\Delta' = \frac{\Delta}{q_L}, \quad Q' = \frac{Q}{q_L},$$

keeping the fixed-point scale within target bounds at the cost of one level.

3.4 CKKS Homomorphic Addition

Component-wise sum Ciphertexts live as pairs (c_0, c_1) , so addition is

$$(c_0, c_1) + (c'_0, c'_1) = (c_0 + c'_0, c_1 + c'_1),$$

preserving the decrypted quantity $c_1 - c_0$ slot-wise.

Scale alignment Addition requires both operands to share the same stored scale Δ ; if not, rescale one ciphertext first (Section 3.3) to keep fixed-point alignment.

Effect on noise The scale remains unchanged and noise growth is negligible, so this is CKKS's cheapest, most stable homomorphic primitive.

2025-11-17

Background of Fully Homomorphic Encryption

- CKKS
- 3.4 CKKS Homomorphic Addition

3.4 CKKS Homomorphic Addition

Component-wise sum Ciphertexts live as pairs (c_0, c_1) , so addition is $(c_0, c_1) + (c'_0, c'_1) = (c_0 + c'_0, c_1 + c'_1)$, preserving the decrypted quantity $c_1 - c_0$ slot-wise.

Scale alignment Addition requires both operands to share the same stored scale Δ ; if not, rescale one ciphertext first (Section 3.3) to keep fixed-point alignment.

Effect on noise The scale remains unchanged and noise growth is negligible, so this is CKKS's cheapest, most stable homomorphic primitive.

“Homomorphic addition in CKKS is the simplest operation. Each ciphertext is a pair (c_0, c_1) , so we just add the two components: $(c_0, c_1) + (c'_0, c'_1) = (c_0 + c'_0, c_1 + c'_1)$. Because the decrypted value is $c_1 - c_0$, the slot-wise sum of plaintext approximations pops out immediately. The only caveat is that both operands must carry the same scale Δ —if they differ, we rescale one ciphertext as described on slide 3.3 so the fixed-point alignment makes sense. Addition doesn't change Δ and only nudges the noise a tiny amount, so it is extremely cheap and stable compared with homomorphic multiplication. This contrast sets up the next slide where multiplication inflates scale and ciphertext size.”

3.5 CKKS Homomorphic Multiplication and Relinearization

Multiplication result Each ciphertext decrypts as $c_1 - c_0s \approx \Delta \cdot m$. Multiplying two ciphertexts therefore expands

$$(c_1 - c_0s)(c'_1 - c'_0s) = d_0 + d_1s + d_2s^2,$$

where

$$d_0 = c_1c'_1, \quad d_1 = -(c_1c'_0 + c_0c'_1), \quad d_2 = c_0c'_0.$$

Why relinearize The appearance of the d_2s^2 term enlarges the ciphertext (now (d_0, d_1, d_2)) and would require the secret key squared to decrypt, increasing storage and computation.

Relinearization Apply the evaluation key $\text{EvalKey}(s^2)$ to fold the d_2s^2 contribution back into the two-term format:

$$(c_0, c_1) \times (c'_0, c'_1) \xrightarrow{\text{multiply}} (d_0, d_1, d_2) \xrightarrow[\text{remove } d_2(s^2)]{\text{relinearize}} (e_0, e_1).$$

Reference CKKS paper (Cheon–Kim–Kim–Song, ASIACRYPT 2017).¹
¹Cheon, Kim, Kim, Song, “Homomorphic Encryption for Arithmetic of Approximate Numbers,” ASIACRYPT 2017.

2025-11-17

Background of Fully Homomorphic Encryption

- CKKS

- 3.5 CKKS Homomorphic Multiplication and Relinearization

“Decrypting a CKKS ciphertext uses $c_1 - c_0s \approx \Delta \cdot m$. When we multiply two ciphertexts we multiply those decrypted forms, so $(c_1 - c_0s)(c'_1 - c'_0s)$ expands to $d_0 + d_1s + d_2s^2$ with $d_0 = c_1c'_1$, $d_1 = -(c_1c'_0 + c_0c'_1)$, and $d_2 = c_0c'_0$. That third term corresponds to an s^2 component, so the ciphertext effectively has three pieces (d_0, d_1, d_2) . If we let those accumulate, ciphertext size and key-switching cost skyrocket because the secret key powers keep growing. Relinearization uses a precomputed evaluation key tied to s^2 to absorb d_2 back into the first two terms, producing a standard two-component ciphertext (e_0, e_1) that decrypts with just s . We always relinearize immediately after multiplication before moving on to rescaling. The full construction lives in the original CKKS paper by Cheon, Kim, Kim, and Song (Asiacrypt 2017) if you want the formal treatment.”

3.5 CKKS Homomorphic Multiplication and Relinearization

Multiplication result: Each ciphertext decrypts as $c_1 - c_0s \approx \Delta \cdot m$. Multiplying two ciphertexts therefore expands

$$(c_1 - c_0s)(c'_1 - c'_0s) = d_0 + d_1s + d_2s^2,$$

where

$$d_0 = c_1c'_1, \quad d_1 = -(c_1c'_0 + c_0c'_1), \quad d_2 = c_0c'_0.$$

Why relinearize: The appearance of the d_2s^2 term enlarges the ciphertext (now (d_0, d_1, d_2)) and would require the secret key squared to decrypt, increasing storage and computation.

Relinearization: Apply the evaluation key $\text{EvalKey}(s^2)$ to fold the d_2s^2 contribution back into the two-term format:

$$(c_0, c_1) \times (c'_0, c'_1) \xrightarrow{\text{multiply}} (d_0, d_1, d_2) \xrightarrow[\text{remove } d_2(s^2)]{\text{relinearize}} (e_0, e_1).$$

Reference: CKKS paper (Cheon–Kim–Kim–Song, ASIACRYPT 2017).¹

¹Cheon, Kim, Kim, Song, “Homomorphic Encryption for Arithmetic of Approximate Numbers,” ASIACRYPT 2017.

4. TFHE Overview

Computation model TFHE operates on encrypted bits; each Boolean value is packed into a (T)LWE sample, unlike CKKS’s approximate real vectors.

Encryption layer LWE-style keys/encapsulation; with $r = 1$ and $m \in \{0, 1\}$:

$$c = (c_0, c_1) = (A, As + \mu m + e),$$

where μ (typically $q/2$) plays the role of the LWE scaling factor Δ .

Logic gates Supports homomorphic AND/XOR/NOT primitives, making it ideal for comparisons, branching, verification, and encrypted control flow.

Bootstrapping Every gate invocation bootstraps away noise, allowing arbitrarily deep Boolean circuits with stable precision.

Contrast to CKKS CKKS excels at numeric workloads but struggles with exact branching or comparisons; TFHE provides the complementary bit-level toolkit.

2025-11-17

Background of Fully Homomorphic Encryption

- TFHE
- 4. TFHE Overview

“Now we pivot to TFHE, which follows a completely different computation model from CKKS. CKKS encrypts approximate real vectors, whereas TFHE encrypts individual bits—each one is a TLWE/LWE sample. The scheme’s core idea is to evaluate Boolean gates such as AND, XOR, and NOT directly on ciphertexts, making it the go-to option for comparisons, conditional checks, verification protocols, and encrypted code execution. CKKS is ill-suited for these bit-level tasks. TFHE reuses the LWE-style key generation and encryption workflow; for intuition we can set $r = 1$ so the ciphertext looks like $(c_0, c_1) = (A, As + \mu m + e)$ just like in Section 2.4, with $m \in \{0, 1\}$ and μ playing the role of the earlier scaling factor Δ . The killer feature is bootstrapping: every gate refreshes the noise budget, so we can evaluate arbitrarily deep logic circuits without the noise blow-up that plagues other schemes. While CKKS also has bootstrapping variants, their noise-refresh performance lags far behind TFHE’s per-gate bootstrap. This bit-level capability fills the gap left by CKKS’s numeric focus.”

4. TFHE Overview

Computation model

TFHE operates on encrypted bits; each Boolean value is packed into a (T)LWE sample, unlike CKKS’s approximate real vectors.

Encryption layer

LWE-style keys/encapsulation; with $r = 1$ and $m \in \{0, 1\}$:

$$c = (c_0, c_1) = (A, As + \mu m + e),$$

where μ (typically $q/2$) plays the role of the LWE scaling factor Δ .

Logic gates

Supports homomorphic AND/XOR/NOT primitives, making it ideal for comparisons, branching, verification, and encrypted control flow.

Bootstrapping

Every gate invocation bootstraps away noise, allowing arbitrarily deep Boolean circuits with stable precision.

Contrast to CKKS

CKKS excels at numeric workloads but struggles with exact branching or comparisons; TFHE provides the complementary bit-level toolkit.

4.1 TFHE Gate Operations

Ciphertext form Encrypt each bit $m \in \{0, 1\}$ as $c = (A, c_1)$ with $c_1 = As + \mu m + e$.

NOT gate Flip the phase to encode $1 - m$; equivalently $c'_1 = As + \mu(1 - m) + e$:

$$\text{NOT}(c) = (-A, \mu - c_1) \bmod q.$$

XOR gate Homomorphic addition implements XOR:

$$\text{XOR}(c_0, c_1) = c_0 + c_1 \pmod q.$$

AND challenge Multiplicative gates inflate scale/noise since

$$(m_0\mu)(m_1\mu) = (m_0m_1)\mu^2,$$

so TFHE uses bootstrapping to refresh ciphertexts after AND/NAND.

2025-11-17

Background of Fully Homomorphic Encryption

- TFHE

- 4.1 TFHE Gate Operations

4.1 TFHE Gate Operations

Ciphertext form Encrypt each bit $m \in \{0, 1\}$ as $c = (A, c_1)$ with $c_1 = As + \mu m + e$.

NOT gate Flip the phase to encode $1 - m$; equivalently $c'_1 = As + \mu(1 - m) + e$:

$$\text{NOT}(c) = (-A, \mu - c_1) \bmod q.$$

XOR gate Homomorphic addition implements XOR:

$$\text{XOR}(c_0, c_1) = c_0 + c_1 \pmod q.$$

AND challenge Multiplicative gates inflate scale/noise since

$$(m_0\mu)(m_1\mu) = (m_0m_1)\mu^2,$$

so TFHE uses bootstrapping to refresh ciphertexts after AND/NAND.

“Each TFHE ciphertext encrypts a bit $m \in \{0, 1\}$ as (A, c_1) with $c_1 = As + \mu m + e$. For NOT we flip the phase: $(1 - m)\mu = \mu - m\mu$, so the refreshed component becomes $c'_1 = As + \mu(1 - m) + e$ and the ciphertext is $(-A, \mu - c_1) \bmod q$. XOR is simply ciphertext addition because the arithmetic is linear, so $\text{XOR}(c, c') = c + c' \pmod q$. AND and NAND are harder: multiplying ciphertexts corresponds to $(\mu m_0)(\mu m_1) = \mu^2 m_0 m_1$, which squares the scale and noise. TFHE combats that by bootstrapping right after multiplicative gates, keeping the noise budget low enough for deep Boolean circuits.”

4.2 TFHE Programmable Bootstrapping (PBS)

- Phase encoding** Each ciphertext hides a phase $\phi \approx \mu m$ (with $\mu \approx q/4$): $\phi \approx 0$ encodes $m = 0$, $\phi \approx \mu$ encodes $m = 1$.
- Bootstrapping goal** Extract the phase homomorphically, evaluate a function on it, and re-encode a fresh low-noise ciphertext without decrypting.
- PBS view** Treat bootstrapping as Decode $\rightarrow$ Evaluate $\rightarrow$ Re-encrypt inside the ciphertext, guided by a bootstrapping key.

2025-11-17

- Background of Fully Homomorphic Encryption
 - TFHE
 - 4.2 TFHE Programmable Bootstrapping (PBS)

“PBS starts from the phase representation: TLWE ciphertexts encode a phase near either 0 or μ depending on the bit. The goal is to extract that phase homomorphically, evaluate the desired Boolean function, and re-encrypt a fresh ciphertext so evaluation can continue. Conceptually it’s a decode–evaluate–re-encrypt pipeline running entirely under encryption.”

4.2 TFHE Programmable Bootstrapping (PBS)

Phase encoding Each ciphertext hides a phase $\phi \approx \mu m$ (with $\mu \approx q/4$): $\phi \approx 0$ encodes $m = 0$, $\phi \approx \mu$ encodes $m = 1$.

Bootstrapping goal Extract the phase homomorphically, evaluate a function on it, and re-encode a fresh low-noise ciphertext without decrypting.

PBS view Treat bootstrapping as Decode $\rightarrow$ Evaluate $\rightarrow$ Re-encrypt inside the ciphertext, guided by a bootstrapping key.

4.2 TFHE Programmable Bootstrapping (PBS)

Phase reconstruction Begin with $\phi = c_1 - A\mathbf{s}$; instead of exposing $\mathbf{s}$, compute

$$\text{Enc}(\phi) = \text{Enc}(c_1 - A\mathbf{s}).$$

Need for $\text{Enc}(A\mathbf{s})$ Since $A\mathbf{s} = \sum_{i=1}^n a_i s_i$,

$$\sum_{i=1}^n a_i \cdot \text{Enc}_{\text{GLWE}}(s_i) = \text{Enc}\left(\sum_{i=1}^n a_i s_i\right) = \text{Enc}(A\mathbf{s}),$$

enabling phase recovery without revealing $\mathbf{s}$.

Bootstrapping key Publish the GLWE encryptions of the secret coefficients:

$$\text{BK} = \{\text{Enc}_{\text{GLWE}}(s_1), \dots, \text{Enc}_{\text{GLWE}}(s_n)\},$$

so PBS has the encrypted ingredients needed to refresh ciphertexts safely.

2025-11-17

Background of Fully Homomorphic Encryption

- TFHE

- 4.2 TFHE Programmable Bootstrapping (PBS)

4.2 TFHE Programmable Bootstrapping (PBS)

Phase reconstruction Begin with $\phi = c_1 - A\mathbf{s}$; instead of exposing $\mathbf{s}$, compute

$$\text{Enc}(\phi) = \text{Enc}(c_1 - A\mathbf{s}).$$

Need for $\text{Enc}(A\mathbf{s})$ Since $A\mathbf{s} = \sum_{i=1}^n a_i s_i$,

$$\sum_{i=1}^n a_i \cdot \text{Enc}_{\text{GLWE}}(s_i) = \text{Enc}\left(\sum_{i=1}^n a_i s_i\right) = \text{Enc}(A\mathbf{s}),$$

enabling phase recovery without revealing $\mathbf{s}$.

Bootstrapping key Publish the GLWE encryptions of the secret coefficients:

$$\text{BK} = \{\text{Enc}_{\text{GLWE}}(s_1), \dots, \text{Enc}_{\text{GLWE}}(s_n)\},$$

so PBS has the encrypted ingredients needed to refresh ciphertexts safely.

“The first PBS step is phase reconstruction. We’d like $\phi = c_1 - A\mathbf{s}$, but naively subtracting $A\mathbf{s}$ would reveal the secret key. Instead we work with $\text{Enc}(\phi) = \text{Enc}(c_1 - A\mathbf{s})$, which means we only need $\text{Enc}(A\mathbf{s})$. Because $A\mathbf{s} = \sum_i a_i s_i$, we can publish GLWE encryptions of each secret coefficient and homomorphically sum $\sum_i a_i \text{Enc}_{\text{GLWE}}(s_i) = \text{Enc}(A\mathbf{s})$. This collection is the bootstrapping key, letting PBS ‘decrypt itself’ just enough to refresh the ciphertext while keeping $\mathbf{s}$ hidden.”

4.2 TFHE Programmable Bootstrapping (PBS)

Encrypted phase Using the previous step we carry forward $\text{Enc}(\phi) = \text{Enc}(c_1 - As)$.

Programmable goal Map ϕ through f so $\text{Enc}(\phi) \rightarrow \text{Enc}(f(b))$.

Gadget decomposition Produce weights via

$$\text{GadgetDecomp}(c = \text{Enc}(\phi)) = (w_0, w_1, \dots, w_k),$$

which form the encrypted selection signals.

LUT for f Store encrypted outputs

$$\text{LUT}_f = (\text{Enc}(f(0)), \text{Enc}(f(1))),$$

generalized to higher arity as needed.

External product Combine weights and LUT to obtain the refreshed ciphertext:

$$\text{Enc}(f(b)) = \text{LUT}_f \star (w_0, \dots, w_k) = \sum_{i=0}^k w_i \cdot \text{LUT}_f^{(i)}.$$

Here each w_i acts as an encrypted selection weight and $\text{LUT}_f^{(i)}$ stores the encrypted function value.

2025-11-17

Background of Fully Homomorphic Encryption

└ TFHE

└ 4.2 TFHE Programmable Bootstrapping (PBS)

“Once we hold $\text{Enc}(\phi) = \text{Enc}(c_1 - As)$, PBS evaluates the requested Boolean function via a programmable LUT. Run gadget decomposition to obtain $(w_0, \dots, w_k)$, keep the encrypted outputs LUT_f , and compute the external product $\text{LUT}_f \star (w_0, \dots, w_k) = \sum_i w_i \cdot \text{LUT}_f^{(i)}$. The w_i are encrypted selection weights while $\text{LUT}_f^{(i)}$ stores the encrypted function value. This finishes the ciphertext transformation: because we can prebuild LUT_f for any Boolean function, PBS is truly “programmable bootstrapping.”

4.2 TFHE Programmable Bootstrapping (PBS)

Encrypted phase Using the previous step we carry forward $\text{Enc}(\phi) = \text{Enc}(c_1 - As)$.

Programmable goal Map ϕ through f so $\text{Enc}(\phi) \rightarrow \text{Enc}(f(b))$.

Gadget decomposition Produce weights via

$$\text{GadgetDecomp}(c = \text{Enc}(\phi)) = (w_0, w_1, \dots, w_k),$$

which form the encrypted selection signals.

LUT for f Store encrypted outputs

$$\text{LUT}_f = (\text{Enc}(f(0)), \text{Enc}(f(1))),$$

generalized to higher arity as needed.

External product Combine weights and LUT to obtain the refreshed ciphertext:

$$\text{Enc}(f(b)) = \text{LUT}_f \star (w_0, \dots, w_k) = \sum_{i=0}^k w_i \cdot \text{LUT}_f^{(i)}.$$

Here each w_i acts as an encrypted selection weight and $\text{LUT}_f^{(i)}$ stores the encrypted function value.